

Proposta Inicial — Plataforma de Gestão de Massa QA UAT4

Objetivo do Documento

Este documento tem como objetivo consolidar a proposta inicial da estrutura de ETL, modelagem do banco de dados e fluxo operacional da futura plataforma de gerenciamento de massa QA do ambiente UAT4.

A proposta busca:

- Centralizar o controle das massas de teste
 - Organizar a utilização entre QA e automações
 - Facilitar manutenção operacional das massas
 - Melhorar rastreabilidade
 - Simplificar consumo via API
 - Estruturar uma base evolutiva para futuras melhorias
-

Contexto Atual

Hoje o processo funciona da seguinte forma:

1. As tribos preenchem planilhas contendo critérios da massa desejada
 2. São realizadas consultas na base produtiva
 3. As contas retornadas são enviadas para a Tsys
 4. A Tsys realiza mascaramento e carga no ambiente UAT4
 5. O retorno ocorre via arquivo CSV
 6. As massas são utilizadas pelos QAs e automações
-

Problemas Identificados

Gestão descentralizada da massa

Atualmente não existe uma visão centralizada de:

- quais massas existem
- quem está utilizando
- quais massas são reutilizáveis
- quais massas estão saudáveis
- quais massas possuem cartões válidos

Concorrência entre uso manual e automação

O mesmo cliente pode ser utilizado simultaneamente por:

- automações
- pipelines
- QA manual

Sem controle centralizado, isso gera:

- conflito de utilização
- quebra de cenários
- inconsistência nos testes

Dependência operacional manual

Grande parte da gestão das massas ocorre de forma manual.

Exemplo:

- reutilização informal de massas
- perda de contexto
- massas abandonadas
- dificuldade de rastrear utilização

Necessidade de manutenção operacional

Algumas massas precisam permanecer saudáveis durante o trimestre.

Exemplos:

- pagamento de faturas
- validação de cartões ativos
- controle de vencimento
- atualização de informações operacionais

Objetivo da Solução

A proposta é construir uma plataforma simples e centralizada para:

- ingestão da massa via CSV
 - enriquecimento via APIs Tsys/internas
 - disponibilização controlada via API
 - controle de utilização
 - manutenção operacional via worker
-

Premissas da V1

Safra trimestral

O ambiente UAT4 possui reset trimestral.

A cada nova carga:

- um novo CSV será processado
 - novas massas serão carregadas
 - massas anteriores deixam de ser válidas operacionalmente
-

Modelo simplificado

A V1 prioriza:

- simplicidade operacional
- clareza de domínio
- facilidade de evolução
- baixo acoplamento

Evitando overengineering inicial.

Parceiros fora da V1

Neste primeiro momento:

- integrações de parceiros não serão consideradas
 - o foco será apenas no domínio principal Tsys/UAT4
-

Arquitetura Proposta

CSV TSYS
↓
ETL
↓
Banco Centralizado
↓
Worker Operacional
↓
API de Consumo
↓
QA / Automação

Fluxo Operacional

1. Importação

O CSV retornado pela Tsys será processado pelo ETL.

O ETL será responsável por:

- leitura do arquivo
 - normalização básica
 - persistência inicial
 - relacionamento cliente/cartões
-

2. Enriquecimento

Após a ingestão inicial, algumas informações serão enriquecidas via:

- APIs Tsys
- APIs internas

Exemplos:

- vencimento de fatura
 - status operacional atualizado
 - cartões válidos
 - informações complementares
-

3. Disponibilização da Massa

A massa será disponibilizada via API.

A API será responsável por:

- busca de massa
 - reserva da massa
 - controle de utilização
 - rastreabilidade
-

4. Utilização da Massa

Existem dois principais consumidores:

QA Manual

Normalmente:

- mantém a posse da massa
 - reutiliza o mesmo cliente
 - não libera explicitamente
-

Automação

Normalmente:

- reserva temporariamente
 - executa o cenário
 - libera ao final da execução
-

5. Worker Operacional

O worker será responsável por:

- manutenção operacional
- enriquecimento de dados
- controle preventivo
- atualização de informações

Exemplos:

- pagamento de faturas
 - validação de cartões
 - atualização de vencimentos
-

Modelagem Inicial do Banco

A proposta inicial foi simplificada para quatro tabelas principais.

Objetivo:

- reduzir complexidade
- acelerar implementação
- permitir evolução futura

1. import_batch

Objetivo

Controlar cada safra trimestral importada.

Estrutura

campo	objetivo
id	identificador interno do lote
batch_name	nome da safra. Ex: 2026Q2
environment	ambiente relacionado
imported_at	data/hora da importação
is_active	identifica a safra vigente
status	status da carga
total_clients	quantidade de clientes
total_cards	quantidade de cartões

Observações

A API sempre deverá trabalhar apenas com o lote ativo.

Exemplo:

WHERE is_active = true

2. client

Objetivo

Entidade principal da plataforma.

Representa o cliente QA operacional utilizado nos testes.

Estrutura

campo	objetivo
id	identificador interno
batch_id	safra vinculada
cpf	login principal
account_number	conta vinculada
account_status	status original Tsys
feature	cenário funcional da massa
has_active_card	indica existência de cartão ativo
invoice_due_date	vencimento operacional
in_use_by	consumidor atual da massa
created_at	criação do registro
updated_at	última atualização

Explicação dos principais campos

cpf

Principal identificador operacional utilizado pelos QAs e automações.

feature

Define o objetivo da massa.

Exemplos:

- invoice_payment
- card_block
- installment
- second_copy

Esse campo será utilizado como principal organizador funcional.

has_active_card

Campo operacional utilizado para facilitar filtros rápidos.

Evita consultas complexas e joins frequentes.

invoice_due_date

Principal dado operacional do worker.

Permite:

- controle preventivo
 - pagamento automático
 - priorização operacional
-

in_use_by

Representa quem está utilizando atualmente a massa.

Exemplos:

- qa_joao
 - automation_runner
 - mobile_pipeline
-

3. card

Objetivo

Representar os cartões vinculados ao cliente.

Relacionamento:

client 1:N card

Estrutura

campo	objetivo
id	identificador interno

client_id	cliente vinculado
card_number	cartão mascarado
bin	primeiros 6 dígitos
last_four	últimos 4 dígitos
expiration_date	validade
is_expired	indica expiração
is_active	elegibilidade operacional
created_at	auditoria

Observações

A separação entre cliente e cartão foi mantida porque:

- um cliente possui múltiplos cartões
 - o CSV possui histórico de cartões
 - evita duplicidade excessiva
 - simplifica consultas futuras
-

4. mass_reservation

Objetivo

Registrar histórico operacional de utilização da massa.

Estrutura

campo	objetivo
id	identificador da utilização
client_id	cliente utilizado
consumer	quem consumiu
started_at	início da utilização
ended_at	fim da utilização
status	status operacional
notes	observações
created_at	auditoria

Observações importantes

QA Manual

Normalmente:

- mantém utilização contínua
- pode não finalizar explicitamente
- reutiliza a massa diversas vezes

Nesse caso:

ended_at pode permanecer nulo

Automação

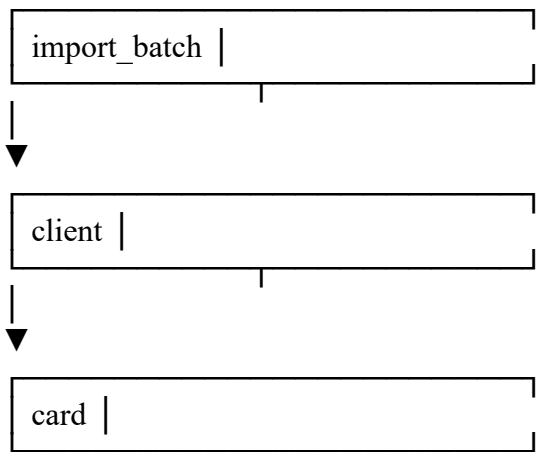
Normalmente:

- inicia utilização
 - executa cenário
 - finaliza utilização
-

Status sugeridos

status
ACTIVE
FINISHED
FAILED
ABANDONED

Diagrama da Estrutura





mass_reservation |

Estratégia de ETL

Origem inicial

CSV Tsys.

Dados provenientes do CSV

Exemplos:

- cpf
 - conta
 - cartões
 - validade
 - status
-

Dados enriquecidos posteriormente

Via APIs Tsys/internas.

Exemplos:

- invoice_due_date
 - status atualizado
 - informações operacionais
 - dados complementares
-

Estratégia de Banco

Tecnologia sugerida

- GCP

- Cloud SQL
 - PostgreSQL
-

Motivos

- excelente suporte relacional
 - consistência transacional
 - facilidade de integração
 - ótimo suporte para APIs
 - facilidade de manutenção
-

Benefícios Esperados

Operacionais

- controle centralizado da massa
 - redução de conflito entre uso manual e automação
 - rastreabilidade operacional
 - organização por funcionalidade
-

Técnicos

- simplificação da gestão da massa
 - facilidade de evolução futura
 - redução de dependência manual
 - melhor observabilidade
-

Arquiteturais

- modelo simples
 - baixo acoplamento
 - fácil manutenção
 - preparado para evolução futura
-

Evoluções Futuras Possíveis

A estrutura atual foi desenhada para permitir evolução gradual.

Possíveis evoluções futuras:

- suporte a parceiros
- múltiplas contas por cliente
- engine avançada de health score
- analytics operacional
- dashboards
- regras inteligentes de recomendação de massa
- catálogo avançado de funcionalidades
- orquestração mais sofisticada de workers

Considerações Finais

A proposta atual busca equilibrar:

- simplicidade
- governança
- rastreabilidade
- operacionalização
- facilidade de evolução

O objetivo da V1 não é resolver todos os cenários futuros, mas criar uma base sólida, simples e sustentável para evolução gradual da plataforma de massa QA.

Estruturei o documento consolidando toda a proposta da V1, incluindo:

- contexto atual
- problemas identificados
- objetivo da solução
- premissas da V1
- arquitetura proposta
- fluxo operacional
- detalhamento completo das tabelas
- estratégia ETL
- estratégia de banco na GCP
- comportamento QA vs automação
- diagramas
- benefícios esperados
- evoluções futuras

O material ficou organizado em formato corporativo, técnico e objetivo para facilitar a apresentação e discussão com o time de dados.

método	endpoint	objetivo
POST	/masses/reserve	reservar massa
POST	/masses/release	liberar massa
GET	/masses	consulta operacional
GET	/masses/{id}	detalhe da massa
GET	/masses/{id}/reservations	histórico

Outro ponto MUITO importante

Na reserva:

a API deve ser transacional

Porque vocês terão concorrência.

Exemplo crítico

Duas automações:

```
buscam massa ao mesmo tempo
```

Sem lock:

```
mesma massa pode ser retornada
```

Então no PostgreSQL

Provavelmente:

```
FOR UPDATE SKIP LOCKED
```

vai ser MUITO importante.

Exemplo conceitual

```
SELECT *  
FROM client  
WHERE in_use_by IS NULL
```

LIMIT 1
FOR UPDATE SKIP LOCKED

Isso provavelmente será:

um dos pontos mais importantes da API.

Minha recomendação arquitetural

A API deve:

controlar toda a concorrência

E nunca deixar:

- automação
- QA
- worker

alterarem o banco diretamente.